value from the query string into the `Message` property.

NOTE You may not see the query string in some browsers (e.g., Opera and Safari). Click the address bar to see the complete URL.

As I mentioned earlier, accessing form values directly from the `Request.Form` or `Request.Query` collections is not recommended within a Razor page. It is all too easy to misspell the index value, so it no longer references a valid form control, or change the name attribute of the control, resulting in the same issue. But there are use cases in which being able to access these collections directly is useful. You saw one such case in chapter 2 when you created the `PassThroughMiddleware` example.

You might also want to access these collections when logging activity within the application, which I cover in more detail in chapter 12. The one thing that is certain about the users of your application is that they will find ingenious ways to use it, well outside of your expectations or imagination. Since you can't stand over their shoulders and watch them all, you need some way to record users' activity, so you can stand some chance of replicating the steps they took in the event they encounter problems. As part of that, you need to gather as much information about the request they made as possible, including the contents of any form submissions. The following listing iterates the form collection and outputs the name of all the keys and their values.

Listing 5.4 Iterating the `Request.Form` collection and outputting the contents

```
<ul>
@if (Request.HasFormContentType)                    ❶
{
    foreach (var item in Request.Form)
    {
```